

PDF to Audiobook

A Deep Dive for the Complete Beginner

Portfolio explainer

The whole project in one paragraph

This is a desktop program that reads the words out of a PDF file and turns them into an mp3 you can listen to. You give it one PDF or a whole batch, optionally check what text it found and narrow it to a page range, pick a language and a regional accent, and press one button. It shows real progress while it works, because it converts the document one page at a time rather than in a single opaque request. It is about 578 lines of Python across two files.

Contents

1	What this project is, and what it is not	2
1.1	The problem it solves	2
1.2	What it deliberately does not do	2
2	Architecture: two files, one clean split	2
2.1	Directory structure	4
3	The converter.py module in detail	4
3.1	The lookup tables at the top	4
3.2	The three reading functions	5
3.3	convert_pdf_to_audio: the heart of the project	6
3.4	Joining the fragments: mp3 concatenation	9
3.5	Finding and using an audio player	9
3.6	The command line entry point	10
4	The main.py module: the graphical interface	10
4.1	Optional drag and drop	10
4.2	QueueItem: one row of state	11
4.3	The widget layout	11
4.4	Queue management	12
4.5	Threads, queues, and the event loop	13
4.6	The message protocol	14
4.7	The verified concurrency bug	15
4.8	Preview and playback	16
5	Dependencies	16
6	What was actually run to verify this document	16
7	Honest findings, collected	17
8	Reading the code in the right order	18

1 What this project is, and what it is not

1.1 The problem it solves

Reading a long PDF on a screen is tiring. If the words could be spoken aloud, you could listen while walking or cooking. Commercial apps do this behind a subscription. This project does it locally with a small Python program.

1.2 What it deliberately does not do

Being precise about the boundaries matters more than a feature list, so here they are up front:

- It does **not** read scanned PDFs. A PDF that is really a photograph of a page contains no text to extract. This project does no OCR (see the jargon box below).
- It does **not** work offline. Speech synthesis is done by Google's servers.
- It does **not** have a volume control, because the speech library it uses does not offer one. The project states this plainly rather than shipping a slider that does nothing.

Jargon: OCR (Optical Character Recognition)

Software that looks at a picture of text and works out which letters it shows. Turning a scan of a page into real, selectable text needs OCR. This project has none, so a scanned PDF yields nothing to read aloud.

Jargon: PDF (Portable Document Format)

A file format for documents that look the same everywhere. Importantly, a PDF is not simply text. It is a set of drawing instructions. A text-based PDF stores the actual characters plus where to draw them, so a program can pull the characters back out. An image-based (scanned) PDF stores only a picture, so there is nothing to pull out.

Jargon: TTS (Text to Speech)

Software that converts written text into spoken audio. This project uses an online TTS service, so it sends text over the internet and receives audio back.

Jargon: mp3

A compressed audio file format. Its key property for this project: an mp3 is a long chain of independent *frames*, each carrying its own small header describing how to decode it. This matters a great deal in Section 3.4.

2 Architecture: two files, one clean split

The project is deliberately split into two modules. The rule is simple: all the real work lives in `converter.py`, and it knows nothing about windows or buttons. `main.py` is only a user interface that calls into it.

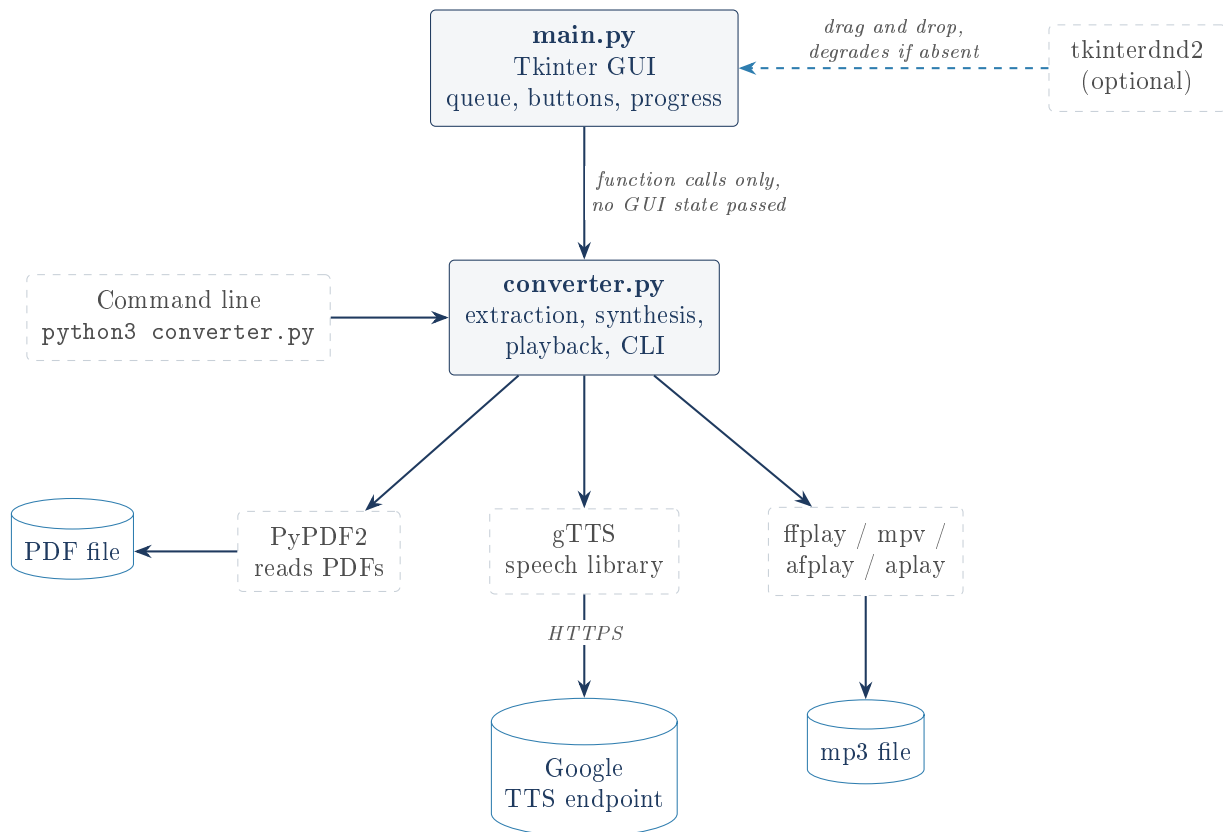


Figure 1: The module split. Solid arrows are hard dependencies; the dashed arrow is an optional one that the code survives without. Note that no arrow ever runs from `converter.py` up into `main.py`: the dependency points one way only.

Why the one-way split is the important design decision

Because `converter.py` never imports Tkinter and never touches a widget, it can be run with no screen at all. That is what makes the command line entry point possible, and it is what let this document's author verify the conversion logic by running it directly (Section 6). A version that mixed the two could only be tested by hand, by clicking. The README records that an earlier pass of this project was exactly that single mixed file, and that splitting it was a deliberate fix.

2.1 Directory structure

```

pdf-to-audiobook
├── converter.py all logic, plus the CLI (218 lines)
├── main.py Tkinter GUI only (360 lines)
├── requirements.txt three pinned dependencies
├── .env.example deliberately empty of variables
├── .gitignore
├── LICENSE PolyForm Strict 1.0.0
├── README.md
└── docs
    ├── screenshots
    │   └── app.png
    └── explainers
        ├── deep-dive.tex this document
        └── brief.tex

```

Figure 2: Every tracked file in the repository. There is no test directory; this is a real gap, discussed in Section 7.

3 The `converter.py` module in detail

3.1 The lookup tables at the top

Three module-level data structures configure everything else.

`LANGUAGES` maps ten human-readable names ("English", "Japanese", and so on) to the short codes the speech library expects ("en", "ja"). The comment in the source is honest that this is a shortlist, not the full set the library supports.

`ACCENTS` maps eight names to internet domain endings such as `com`, `co.uk` and `co.in`. This looks odd until you know the trick behind it, which is worth its own box.

Why an accent is chosen with a website address

The speech library talks to Google Translate’s speech endpoint. Google runs that service on many regional domains, and each region’s voice has that region’s accent. Asking `translate.google.co.uk` gives British-accented English; asking `translate.google.co.in` gives Indian-accented English. So the “accent” setting is literally just a choice of which country’s Google domain to send the request to. The library calls this the `tld`, short for *top-level domain*, the last part of a web address.

`_PLAYER_COMMANDS` is a list, and being a list rather than a dictionary is meaningful: it is an ordered preference. It holds four audio players with the exact command-line arguments each needs to play a file quietly and then exit:

Order	Player	Flags used, and why
1	ffplay	-nodisp (no window), -autoexit, -loglevel quiet
2	mpv	--no-video, --really-quiet
3	afplay	none needed (this is the macOS built-in)
4	aplay	-q for quiet (this is the Linux ALSA built-in)

3.2 The three reading functions

`get_page_count(pdf_path)` opens the PDF and returns how many pages it has. It is used by the GUI to fill in the Pages column.

`extract_text_from_pdf(pdf_path, start_page, end_page)` returns all the text of a page range as one single string, joined with newlines. It is used by the preview window.

`extract_pages_from_pdf(pdf_path, start_page, end_page)` returns a *list*, one string per page. It is used by the conversion routine.

The last two look redundant but are not: the difference between one joined string and a list of per-page strings is precisely what makes progress reporting possible, as Section 3.3 explains.

Both share the same page-range arithmetic, and it is worth reading closely because it contains a real defect:

```

1 total = len(reader.pages)
2 first = 1 if start_page is None else max(1, start_page)
3 last  = total if end_page is None else min(total, end_page)
4 pages_text = []
5 for i in range(first - 1, last):
6     page_text = reader.pages[i].extract_text() or ""
7     pages_text.append(page_text)

```

Listing 1: The range clamping, from converter.py

Line by line: `start_page` and `end_page` are *1-based* (page one is called 1, as a human would say it) and *inclusive* (both ends are converted). Passing nothing for either end means “from the beginning” or “to the end”. The `max(1, ...)` stops a caller asking for page zero or a negative page. The `min(total, ...)` stops a caller running off the end of the document. The `- 1` in `range(first - 1, last)` converts the human 1-based number to the 0-based index Python lists use.

Jargon: 0-based versus 1-based counting

Python numbers the items of a list starting at 0, so the first page is `reader.pages[0]`. Humans number pages starting at 1. Any code that touches both conventions must convert between them, and off-by-one mistakes here are so common they have their own name: *off-by-one errors*.

The `or ""` at the end of the extraction line is small but load-bearing.

Why `or ""` is not decoration

PyPDF2 returns `None` (Python’s “no value at all”) for a page it cannot read text from, which is the normal outcome for a scanned page. Writing `"\\n".join(...)` over a list containing `None` raises a `TypeError` and the program dies. The expression `page.extract_text() or ""` means “use the extracted text, but if it is `None` or empty, use an empty string instead”. One tiny idiom converts a crash into a blank page. The README lists this as a challenge that was actually hit.

Verified defect: an inverted or out-of-range page range fails with a misleading message

The clamping above handles each end independently, and never checks that `first <= last`. Ask for pages 3 to 1, and `range(2, 1)` is empty, so you get no text. Ask for page 99 of a 3-page document and `first` is 99, `last` is 3, and `range(98, 3)` is likewise empty.

Both cases were confirmed by running the real code against a generated 3-page PDF: each returned an empty string, and each drove the command line entry point into `ValueError: No extractable text found in '...' for the selected page range`. That message is wrong about the cause. The document is full of extractable text; the *range* is the problem. A user could reasonably conclude their PDF was a scan.

The GUI is insulated from this by luck rather than by design: its Save Page Range button applies `min/max` to the two spinboxes, so it cannot submit an inverted range. The command line has no such guard. The honest fix is a check in `convert_pdf_to_audio` that raises a message naming the real problem.

3.3 `convert_pdf_to_audio`: the heart of the project

This is the function everything else exists to serve. Its signature takes the PDF path, an optional output directory, the language and accent codes, a `slow` flag, the page range, and an optional `progress_callback`.

Jargon: Callback

A function you hand to another function so it can call you back while it works. Here the caller passes `progress_callback`, and the converter calls it after each page with the numbers “done” and “total”. The converter does not know or care what happens inside it. The GUI passes one that posts a progress message; the command line passes one that prints `Page 2/3`; a test can pass none at all, and the parameter defaults to `None` so nothing is called.

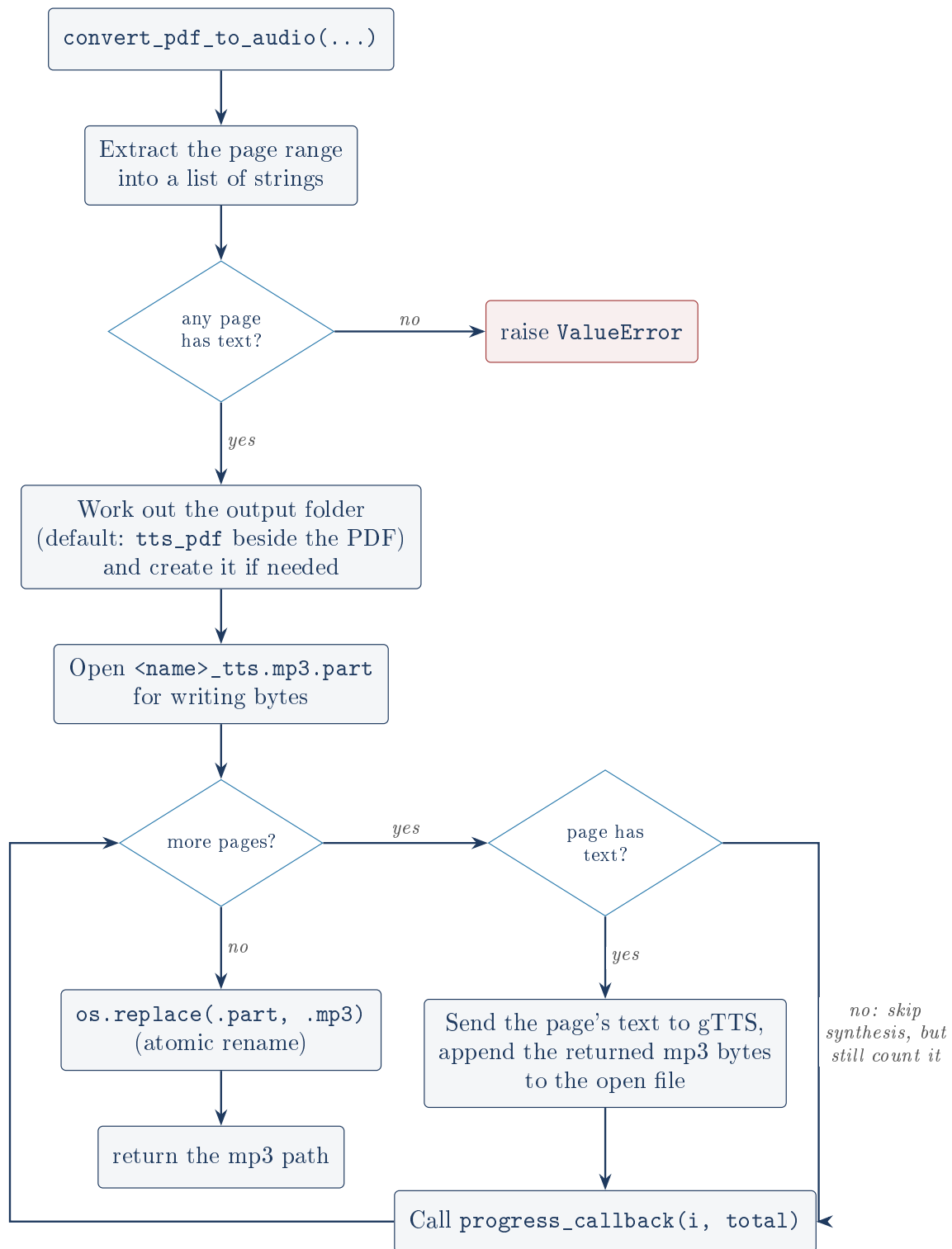


Figure 3: The control flow of `convert_pdf_to_audio`. The loop body is the whole point: one network request per page, and a progress call after every page including the blank ones.

Four decisions in this function deserve individual attention.

3.3.1 Fail early on an empty document

Before creating any folder or file, the function filters the extracted pages down to those with actual content (`[p for p in pages if p.strip()]`) and raises a `ValueError` if none remain. `strip()` removes surrounding whitespace, so a page holding only spaces and newlines correctly counts as

empty. This ordering means a document with nothing to say never leaves an empty mp3 or a stray folder behind.

3.3.2 Where the output goes

If the caller gives no `output_dir`, the function builds one: a folder named `tts_pdf` sitting next to the source PDF. The output file is the PDF's own name with `_tts.mp3` appended, so `report.pdf` becomes `tts_pdf/report_tts.mp3`.

Same range, same name: a silent overwrite

The output name is derived only from the PDF's filename. Convert pages 1 to 5 of `report.pdf`, then convert pages 6 to 10 of the same file, and the second run overwrites the first without warning. The page range is not part of the name. For a tool whose headline feature is page ranges, this is a sharp edge. Including the range in the filename when one is given would cost one line.

3.3.3 The `.part` file and the atomic rename

The function does not write to `report_tts.mp3` directly. It writes to `report_tts.mp3.part`, and only when the whole document has been synthesized does it call `os.replace` to rename that into place. A `try/finally` block deletes the `.part` file if anything went wrong.

Why write to a temporary name first

If the network drops halfway through page 7 of 20, a program writing directly to `report_tts.mp3` leaves behind a real-looking mp3 containing six and a half pages. Nothing marks it as broken. By writing to `.part` and renaming only on success, the final name either does not exist or is complete: there is no in-between state. On the same filesystem `os.replace` is *atomic*, meaning the operating system guarantees it either happens completely or not at all. This is a well-known professional pattern, and finding it in a small hobby-scale utility is a genuinely good sign.

3.3.4 One request per page, not one per document

This is the design insight the whole project turns on.

Why per-page synthesis is what buys real progress

The obvious implementation is `gTTS(text=whole_document).save(path)`: one call, one network request. But that call is a black box. It takes a minute, says nothing, and finishes. There is no hook inside it to report progress, so any progress bar drawn around it would be a lie, an animation on a timer.

Splitting the work per page changes the arithmetic. Twenty pages become twenty requests, and the code regains control between each one. That is where `progress_callback` is called, and it is why the queue can honestly say "Converting page 7/20". The progress is real because the boundaries are real.

The cost is honest too: twenty small requests are slower than one large one, and twenty chances to hit a rate limit. The project accepts that trade for truthful progress.

Note the loop calls `progress_callback(i, total)` for *every* page, but only calls `gTTS` for pages that have text. A blank page advances the counter instantly. That is the right choice: progress should track pages consumed, not requests made.

Progress counts positions in the range, not page numbers

`total` is `len(pages)`, the number of pages *in the selected range*, and `i` counts from 1 within that range. So converting pages 5 to 7 of a book displays “Converting page 1/3”, not “page 5/7”. The fraction is correct; the labels are misleading whenever a range starts anywhere but page 1. The status text says “page”, which is exactly the word that makes a reader expect the document’s page number.

3.4 Joining the fragments: mp3 concatenation

Each page produces its own small mp3. They must become one file. The code does the simplest possible thing: it keeps a single file handle open and asks gTTS to write each page’s bytes into it, one after another (`tts.write_to_fp(out)`).

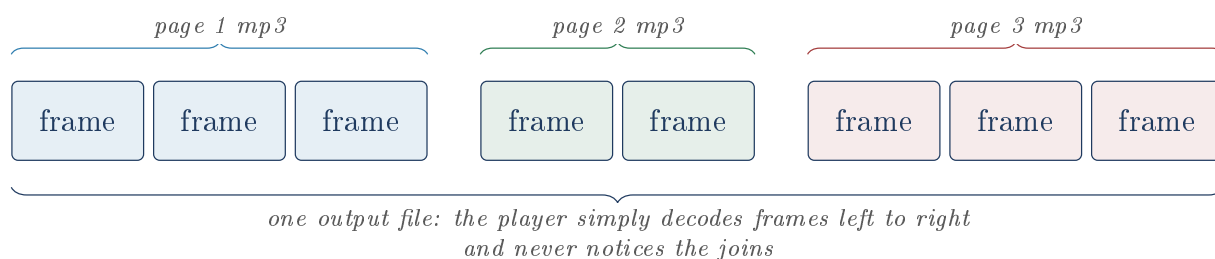


Figure 4: Why raw byte concatenation produces a playable mp3. Every frame carries its own header, so a decoder needs no global index and no file-level structure.

Why this works, and why it is still a little informal

Most file formats would be destroyed by this. Gluing two PNG files together does not produce a taller image; it produces a corrupt file, because a PNG has one header describing the whole picture. An mp3 is different: it is a stream of self-describing frames. A decoder finds a frame header, decodes that frame, and looks for the next one. So concatenated mp3s just play, one after another.

The caveat, which the README states itself: this relies on a property of the format rather than on a documented promise from any specification. Doing it “properly” would mean adding a dependency such as `pydub` or shelling out to `ffmpeg` to perform the same byte copy with more ceremony. The project made a reasoned trade and wrote down why.

This document did not take that on trust. Section 6 reports the result of decoding a real concatenated file.

3.5 Finding and using an audio player

Python’s standard library has no way to play audio. So `find_audio_player` walks `_PLAYER_COMMANDS` in order and uses `shutil.which` to ask “is this program installed on this machine?”, exactly as a shell does when you type a command. It returns the first one found, or `None`.

`play_audio(path)` calls it, and if nothing was found it returns `False` immediately. Otherwise it runs the player with `subprocess.run(...)` and waits for it to finish, then returns `True`.

A boolean return is an honesty mechanism

`play_audio` returning `True/False` rather than just trying and shrugging is the whole reason the GUI can tell the user “no player found, here is the path, open it yourself”. The README records that an early draft simply did nothing silently on such a machine. A function that reports its own failure lets every caller decide what to say. The `check=False` in

the `subprocess.run` call means a player that exits with an error will not raise an exception; the project treats launching the player as the success condition, not what the player then does.

3.6 The command line entry point

Guarded by `if __name__ == "__main__":`, the bottom of `converter.py` builds an `argparse` parser exposing every option: `-o/--output-dir`, `-l/--lang`, `-t/--tld`, `--slow`, `--start-page`, `--end-page` and `--play`. Its progress callback prints `Page 2/3` using a carriage return so the line rewrites itself in place instead of scrolling.

Jargon: `if __name__ == "__main__"`

A standard Python idiom meaning “only run this when the file is executed directly, not when another file imports it”. Without it, merely importing `converter` from `main.py` would try to parse command line arguments and crash the GUI.

The CLI reports user mistakes as raw crashes

Confirmed by running it: pointing the command line at a file that does not exist prints a bare `FileNotFoundError` traceback, and an inverted page range prints a bare `ValueError` traceback. A traceback is a developer’s diagnostic, not a user message. Wrapping the call in a `try/except` that prints one clear line and exits with a non-zero status is a few lines of work and is what separates a script from a tool. The GUI, by contrast, does catch exceptions per file and surfaces them in the queue.

4 The `main.py` module: the graphical interface

4.1 Optional drag and drop

The very first thing of interest is a conditional import:

```
1 try:
2     from tkinterdnd2 import DND_FILES, TkinterDnD
3     DND_AVAILABLE = True
4 except ImportError:
5     DND_AVAILABLE = False
```

Listing 2: Optional dependency handling, from `main.py`

Graceful degradation

`tkinterdnd2` bundles a native extension and does not install cleanly everywhere. If it is missing, this code sets a flag instead of crashing. Later, the flag decides three things: whether the window registers a drop target, which subtitle text is shown (the fallback version says plainly that drag and drop is unavailable and why), and whether the main window is a `TkinterDnD.Tk` or a plain `tk.Tk`. The feature disappears; the app does not. This is called *graceful degradation*, and the honest subtitle is a nice touch: the user is told what is missing rather than left wondering.

Note for anyone reading this document as verification: `tkinterdnd2` is *not* installed on the machine where these checks were run, so the drag-and-drop path is unexercised here. The fallback path is the one that ran.

4.2 QueueItem: one row of state

A tiny class holding everything known about one queued PDF: its `path`, its `name` (just the filename, for display), its `page_count`, the chosen `start_page` and `end_page`, a `status` string, and the `output_path` once converted.

Its constructor calls `get_page_count` inside a `try/except` that catches every exception and falls back to `page_count = 0`, so one unreadable PDF cannot stop a batch from being queued.

The bare except Exception hides why a PDF failed

`QueueItem.__init__` catches every possible error and records only “0 pages”. An encrypted PDF, a corrupt file, and a file that is not a PDF at all are collapsed into the same silent outcome: the queue shows ? pages and range n/a. The user is never told which of those happened, and the real exception is discarded, not logged. The failure only surfaces later, at conversion time, as the misleading “No extractable text” message from Section 3.3. Keeping the exception text and showing it in the Status column would cost almost nothing.

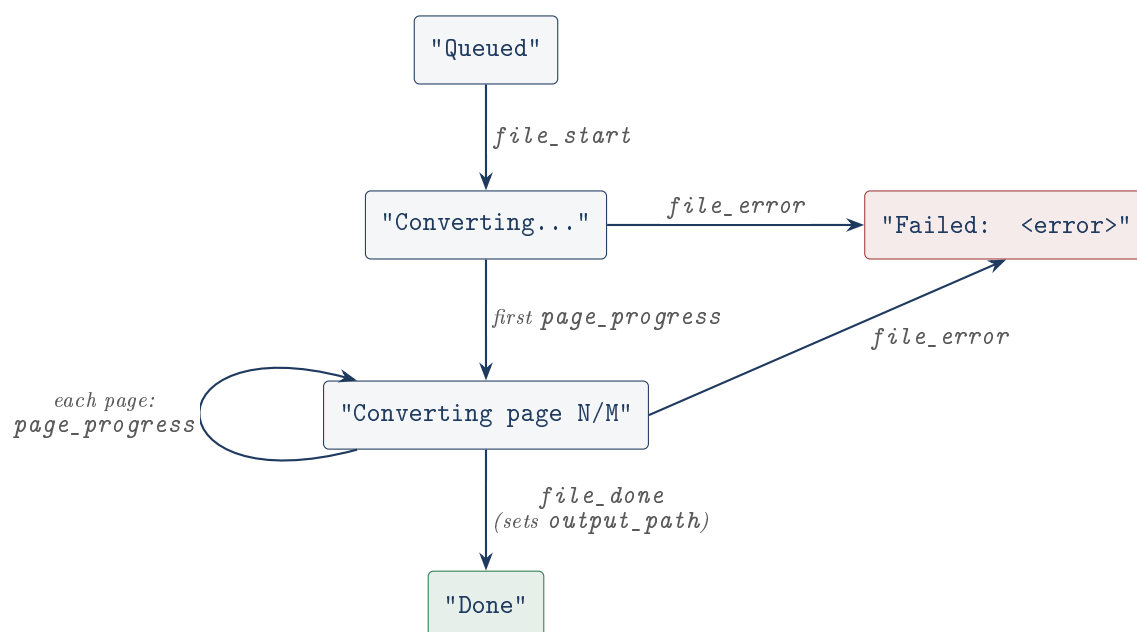


Figure 5: The status of a single `QueueItem` over its life. These are plain strings written straight into the Status column, not an enumeration; nothing in the code constrains them.

4.3 The widget layout

`App._build_widgets` constructs the window top to bottom: a title, the honest subtitle, the queue table, a row of five buttons, a Voice options panel, the Convert All button with the overall progress bar beside it, and a status label at the bottom.

The queue is a `ttk.Treeview`, Tk’s table widget, with a tree column for the filename and three value columns: Pages, Range, Status.

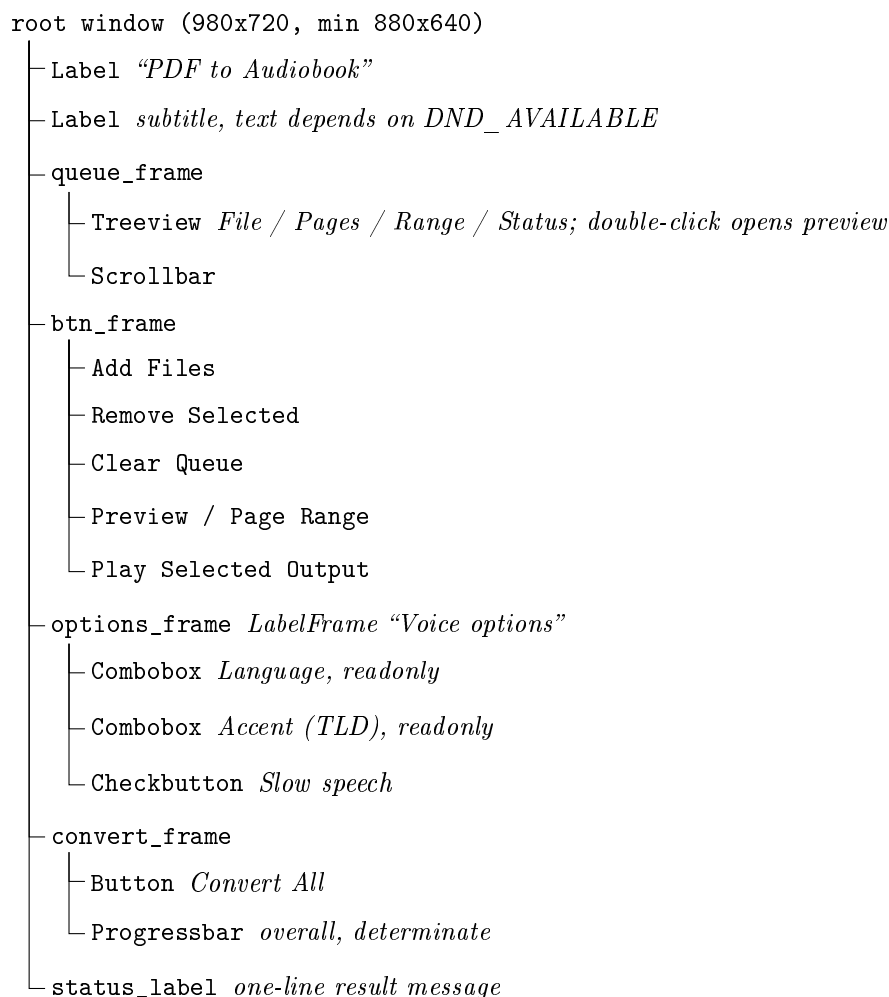


Figure 6: The widget tree. Both comboboxes are `state="readonly"`, so the user picks from the list and cannot type a nonsense value.

The two voice dropdowns are independent, so nonsense combinations are reachable

Language and Accent are separate controls with no relationship in the code: `lang = LANGUAGES[...]` and `tld = ACCENTS[...]` are read independently in `start_conversion`. Nothing stops a user selecting Japanese with the Ireland accent or Hindi with the South Africa accent. The regional domains carry meaningful accents for English and very little for anything else, so most of the 80 available combinations are meaningless. The UI presents all of them as equally valid. Greying out accents for non-English languages, or labelling the control “English accent”, would set honest expectations.

Smaller and in the same area: `ACCENTS` lists both “Default (.com)” and “US (.com)”, which map to the identical value “com”. Two menu entries do exactly the same thing.

4.4 Queue management

`add_files` opens a native multi-select dialog filtered to `*.pdf`. `_on_drop` handles a drop event, splits the platform-specific string of paths with `self.root.tk.splitlist`, and keeps only names ending in `.pdf`, reporting plainly if the drop contained none.

Both funnel into `_add_paths`, which de-duplicates against paths already queued by building a set first, counts what it added, and reports either “N file(s) queued” in green or “Those file(s) are already in the queue” in red.

`_refresh_tree` rebuilds the whole table from `self.items`. It is the single place the display is

regenerated, and it sets each row's `iid` (Tk's identifier for a row) to the item's position in the list, as a string.

Using the list position as the row identifier is the root of the bug in Section 4.7

`_refresh_tree` assigns `iid=str(i)`, and `remove_selected` and `_selected_item` convert that identifier straight back into a list index with `int(iid)`. The identifier is therefore not stable: it is a position that changes whenever the list changes. Remove the first of three files and the third item's identifier silently changes from "2" to "1". Nothing else in the program knows that happened. A stable identity, the file path or a counter that never repeats, would remove a whole class of error.

4.5 Threads, queues, and the event loop

This is the most technically interesting part of the file, and it needs three concepts first.

Jargon: Event loop

A GUI program spends its life in a loop: wait for something to happen (a click, a keypress), run the code for it, repeat. In Tkinter this loop is started by `root.mainloop()`. Crucially, while your code runs, the loop is not looping, so nothing gets redrawn. Any slow work done directly in a button handler freezes the window.

Jargon: Thread

A second line of execution inside the same program, running at the same time as the first. Putting slow work on a background thread lets the event loop keep spinning, so the window stays alive.

The rule that shapes this entire design: Tkinter is not thread-safe

Widgets may only be touched from the thread that created them. A background thread that calls `label.config(...)` directly may work, may garble the display, or may crash the process, unpredictably. So the worker thread cannot update the queue itself. It needs a way to hand facts to the main thread and let the main thread do the drawing. That way is a `queue.Queue`.

Jargon: `queue.Queue`

A list designed to be safely shared between threads. One thread puts items in, another gets them out, and the class handles all the locking internally. It is the standard, boring, correct answer to "how do two threads talk to each other in Python".

The mechanism has three parts working together:

1. `start_conversion` (main thread) reads the dropdown values, disables the Convert button, sets the progress bar's maximum to the number of files, and starts a `threading.Thread` running `_convert_worker`. It is a *daemon* thread, meaning it will not keep the program alive if the user closes the window.
2. `_convert_worker` (worker thread) loops over the files, calls `convert_pdf_to_audio`, and puts messages onto `self.progress_queue`. It touches no widget.
3. `_poll_progress_queue` (main thread) drains the queue, updates the widgets, and then uses `self.root.after(100, self._poll_progress_queue)` to schedule itself to run again in 100 milliseconds. It is started once in the constructor and re-arms itself forever after.

Jargon: `root.after(ms, func)`

Tkinter's timer: "run `func` in `ms` milliseconds, from the main thread". A function that ends by calling `after` on itself creates a repeating poll. Note it schedules *once*: if a run of the function ends without reaching that line, the chain stops permanently. That fact is the whole of Section 4.7.

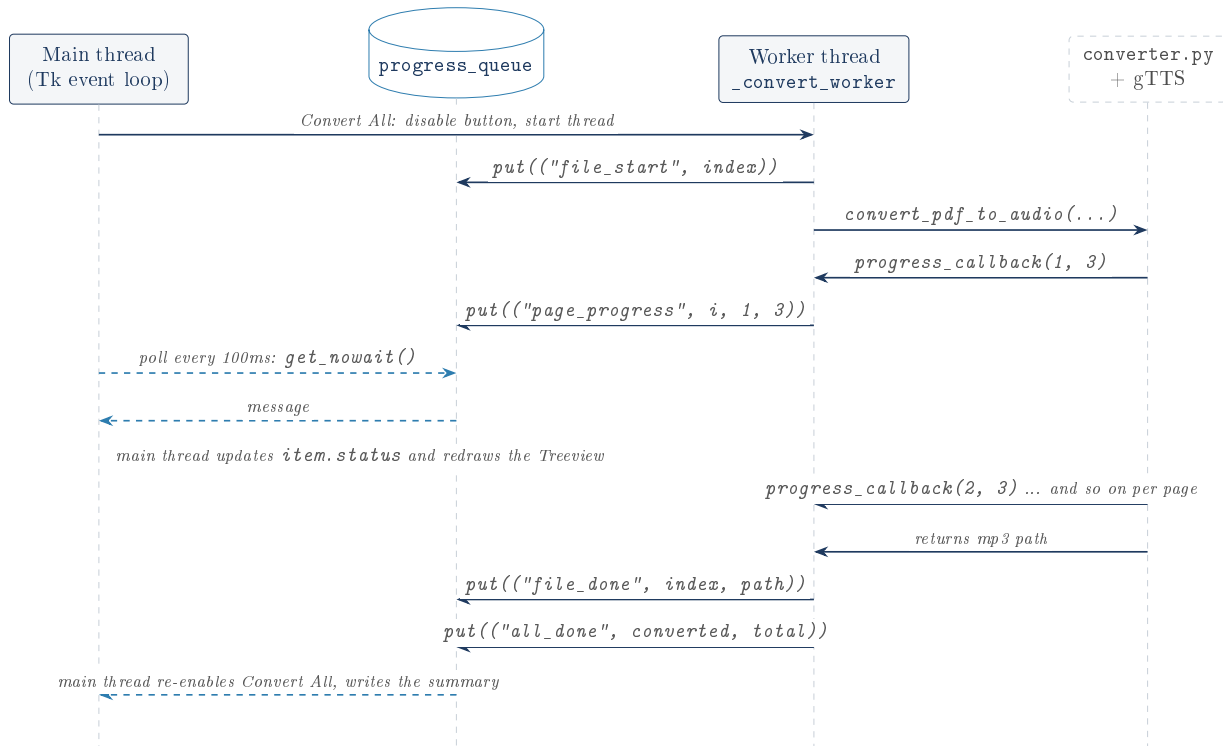


Figure 7: The full conversion sequence. The queue is drawn as its own lifeline for a reason: no arrow ever runs directly from the worker to the main thread. Every fact the worker learns reaches the widgets through the queue, which is exactly what makes the design safe.

4.6 The message protocol

The two threads communicate with plain tuples whose first element names the message. This is an informal protocol, but a real one:

Message	Payload	What the main thread does
<code>file_start</code>	<code>index</code>	status becomes "Converting..."
<code>page_progress</code>	<code>index, done, total</code>	status becomes "Converting page N/M"
<code>file_done</code>	<code>index, output_path</code>	status "Done", store path, bar +1
<code>file_error</code>	<code>index, error text</code>	status "Failed: ...", bar +1
<code>all_done</code>	<code>converted, total</code>	re-enable the button, write the summary

The polling function drains with `get_nowait()` inside a `while True`, and the loop is escaped by catching `queue.Empty`. Using the exception as the exit condition is idiomatic Python here, not a smell.

The worker wraps each file in its own `try/except Exception`, so one failed PDF sends a `file_error` and the batch continues to the next file. The overall bar advances for failures as well as successes, which is right: the bar tracks files *attempted*. At the end, `all_done` carries the counts, and the summary distinguishes all-succeeded, some-failed, and all-failed.

One subtlety worth pointing out, because it is a classic Python trap that this code gets right:

```

1 def on_page(done, total, index=index):
2     self.progress_queue.put(("page_progress", index, done, total))

```

Listing 3: The default-argument capture, from main.py

Why `index=index` is necessary and not redundant

A function defined inside a loop that refers to the loop variable does not remember the value the variable had when the function was created; it looks the variable up when it is *called*. By then the loop has moved on, and every callback would report the last index. Binding it as a default argument (`index=index`) captures the value at definition time. The code does this correctly. It is a small detail that a lot of otherwise-good Python gets wrong.

4.7 The verified concurrency bug

Verified, serious: pressing Clear Queue or Remove Selected during a conversion permanently freezes the interface

This was reproduced by running the real App class against a real Tk window, not deduced from reading.

The mechanism. `start_conversion` disables only the Convert All button. Add Files, Remove Selected and Clear Queue all stay live while the worker runs. `clear_queue` sets `self.items = []`. The worker, meanwhile, is iterating the *old* list object and keeps posting messages carrying indices into it. The next poll executes `self.items[index]` against the now-empty list and raises `IndexError`.

The consequence is what makes this severe rather than cosmetic. The only `except` in `_poll_progress_queue` catches `queue.Empty`. An `IndexError` propagates out of the function, so execution never reaches the final line, `self.root.after(100, self._poll_progress_queue)`. The self-rescheduling chain breaks and is never restarted. The polling loop is dead for the rest of the session.

Observed, with a withdrawn Tk window and a captured exception handler:

- **errors seen:** `[('IndexError', 'list index out of range')]`
- A message posted afterwards was never consumed: the queue never drains again, confirming the loop is gone.
- **convert button state after crash:** disabled, and it stayed disabled even after an `all_done` message was queued, because nothing is left alive to read it.

So the user sees: a frozen queue that never updates again, a stalled progress bar, and a Convert All button greyed out forever. Only restarting the app recovers. The worker thread, meanwhile, carries on to completion in the background and writes its mp3 files successfully, which the user has no way of learning from the interface.

`remove_selected` is the same fault with a quieter face: it removes items and re-indexes the survivors, so in-flight messages either hit the wrong row (a status written onto an unrelated file) or run off the end and kill the loop identically.

Two independent fixes, either sufficient: disable the queue-mutating buttons for the duration of the conversion, exactly as Convert All already is; or make messages carry a stable identifier (the file path, or the `QueueItem` itself) instead of a list position. Wrapping the poll body in a broad `try/except` that still re-arms the timer would prevent the permanent freeze, but it treats the symptom. This is the single most valuable finding in this document.

4.8 Preview and playback

`open_preview` builds a `Toplevel` (a second window) with two spinboxes for the range, a text area, a Load Preview Text button and a Save Page Range button. When the range yields nothing it shows an explicitly helpful message naming the likely cause: a scanned or image-only page. `save_range` applies `min/max` to the two spinbox values, which as noted is what keeps the GUI clear of the inverted-range defect.

`play_selected` refuses politely if nothing is selected or the file is not converted yet, then runs `play_audio` on *another* background thread, because that call blocks until the audio finishes. If no player was found, it hops back to the main thread with `self.root.after(0, ...)` to show the message box. That detail is the thread-safety rule from earlier being applied correctly a second time, in a place where it would have been easy to forget.

5 Dependencies

Package	Pin	Role
PyPDF2	3.0.1	Reads PDFs; provides <code>PdfReader</code> , <code>.pages</code> and <code>.extract_text()</code> .
gTTS	2.5.4	Sends text to Google Translate's speech endpoint and returns mp3 bytes.
tkinterdnd2	0.6.2	Optional. Adds drag and drop. Bundles a native Tk extension, which is why it is treated as optional.

Tkinter itself is not listed because it ships with Python. All three are pinned to exact versions with `==`, which is the right call for an application: it means a fresh install today behaves like the one the author tested.

The dependency that defines the project's limits

gTTS is not a speech engine. It is a thin client for a Google web endpoint. Everything that follows from that is unavoidable: the tool needs an internet connection, it is subject to rate limits, it depends on an endpoint Google never promised to keep stable, and it is the reason there is no volume control. The README says all of this in bold rather than burying it. `.env.example` contains no variables at all, and says so: there is no API key, because the endpoint needs none.

6 What was actually run to verify this document

Nothing in the sections above was taken on trust where it could be executed. On the machine where this was written, PyPDF2 3.0.1, gTTS and a working network were available; `tkinterdnd2` was not. A 3-page PDF and a 2-page text-free PDF were generated for the purpose.

Claim tested	Result
<code>get_page_count</code>	Returned 3 for the 3-page PDF.
Page range extraction	Range 2 to 3 returned exactly pages 2 and 3, inclusive at both ends as documented.
Blank document handling	The text-free PDF raised the expected <code>ValueError</code> rather than crashing or writing an empty mp3.
Inverted range 3 to 1	Returned an empty string, then the misleading <code>ValueError</code> . Defect confirmed.
Out-of-range start 99	Same as above. Defect confirmed.
<code>find_audio_player</code>	Found <code>ffmpeg</code> first, as the ordering intends.
Full conversion	Succeeded. Progress fired 1/3, 2/3, 3/3. Output 77,952 bytes.
mp3 concatenation	<code>file</code> reports MPEG ADTS, layer III, v2, 64 kbps, 24 kHz, Monaural. <code>ffmpeg</code> decoded the whole file to WAV with no errors . The concatenation claim holds.
Range vs full duration	Full 3 pages: 9.744 s . Pages 2 to 3: 6.576 s . That is exactly 3:2, or 3.248 s per page. The page range genuinely controls what is synthesized.
GUI freeze bug	Reproduced against a real App and a real Tk root. See Section 4.7.
CLI error handling	Missing file and inverted range both print raw tracebacks.

The duration ratio deserves a note: it is not merely “smaller, therefore probably right”. Three equal-length pages producing 9.744 s and two of them producing 6.576 s is an exact 3:2 with a per-page figure that matches to the millisecond. That is strong evidence the concatenation drops nothing and duplicates nothing.

7 Honest findings, collected

Everything below is stated in full in its own section above; this is the summary a reader should leave with, worst first.

1. **The queue-mutation freeze (Section 4.7).** Verified by execution. Clear Queue or Remove Selected during a conversion kills the progress polling loop permanently and leaves Convert All disabled forever. This is a bricked interface from a single ordinary click, and it is the one thing here worth fixing first.
2. **Row identity is a list position.** The direct cause of the above, and a source of silently mislabelled rows even when it does not crash.
3. **Inverted and out-of-range page ranges produce a message blaming the document.** Verified. Only the GUI’s min/max accident keeps users away from it; the CLI is fully exposed.
4. **The CLI prints tracebacks at users.**
5. **Silent overwrite when converting two different ranges of one PDF.** The output name ignores the range.
6. **Progress labels say “page N/M” but count positions within the range,** so a range starting at page 5 reports page 1.
7. **QueueItem swallows every exception** and reduces encrypted, corrupt and non-PDF files to an indistinguishable ?.

8. **Language and accent are independent**, so most of the 80 combinations the UI offers are meaningless, and ACCENTS contains a duplicate.
9. **No automated tests at all**. There is no test directory. Every check in Section 6 was written from scratch for this document and thrown away. The clean module split means a pytest suite over `converter.py` would be straightforward; its absence is a choice, not an obstacle. The README lists this itself under “What I’d do differently”.
10. **Offline it does nothing**. Correctly and prominently documented, but it remains the largest functional limit.
11. **Raw mp3 concatenation rests on a format property, not a specification guarantee**. It verified clean here. The README already flags it.

The fair verdict

The bones of this project are better than its size suggests. The one-way module split, the atomic `.part` rename, the queue-based thread communication, the default-argument capture, the boolean-returning `play_audio`, and the refusal to ship a fake volume slider are all decisions a careful engineer makes. The README is unusually honest, including about the things this document confirms.

The weaknesses cluster in one place: the boundary between the GUI’s data model and the worker thread, where positions are used as identities and only one of four dangerous buttons is disabled. That is where the only severe bug lives, and it is a contained, well-understood fix rather than a design flaw. The other findings are polish.

8 Reading the code in the right order

For someone opening the repository for the first time:

1. `converter.py`, the three lookup tables, to see what is configurable.
2. `extract_pages_from_pdf`, the shortest function that matters.
3. `convert_pdf_to_audio`, top to bottom. This is the project.
4. The `__main__` block, to see the whole thing driven with no GUI at all.
5. `main.py`, `QueueItem` first, then `_build_widgets`.
6. `start_conversion`, `_convert_worker` and `_poll_progress_queue` as one unit. They only make sense together, and they are where the interesting bug lives.